

CHAPTER

8

Specification Techniques

8.1 INTRODUCTION

Specification of a system is the first step towards its realization. After performing the requirement analysis, the specification process starts. Since an embedded system consists of a set of sequential/parallel subtasks, their specification should also be able to capture this. The interaction between the tasks be clearly spelt out and at the same time, the technique be easy to write, understand and verify the system requirements. There are many specification methodologies that have been developed, however, before going to these, let us have a look at the desirable features of any specification technique.

1. A specification technique should have the ability to describe a system from various angles, such as timing, state-oriented and event-oriented approaches. Timing is one of the most important issues in embedded system design. On the other hand, many of the control-dominated applications can most conveniently be specified using a state-oriented approach. Finally, the embedded systems are generally reactive in nature, responding to the events occurring in the environment. Thus, the ability to specify the system from an event-oriented approach is also a desirable feature of the specification technique.
2. Support for hierarchy is another very important feature. In the absence of hierarchy, the specification may become too big to handle either manually or electronically. Thus, usage of hierarchy, concealing the details at higher levels, can effectively make the specification more amenable for understanding and design.
3. Concurrency, synchronization and communication are the three essential aspects of distributed embedded systems. The specification technique must support these aspects efficiently.
4. It is also essential to verify an embedded system specification for its correctness. For doing this, it is desirable that the specification be executable. Thus, a technique based on some programming language appears to be a better choice from this perspective.
5. Automated synthesis is another important desirable feature of embedded system specification. This means full- or semi-automatic tools or techniques to convert a specification into a design.

6. Feature should also be present to specify non-functional quantities, like size, weight, fault-tolerance, user-friendliness, power consumption, life-time, etc., of the system to be designed.

The specification techniques reported in the literature may have one/more features missing from the previous list. The techniques are built around some *models of computation* (MOC). One of the very important MOCs is due to *Von Neumann*. However, this model does not have any notion of time built with it. Thus, it is not much suitable for modelling embedded systems. For this, many other models have been developed that can effectively capture timing and concurrency.

The following are some of those models:

- ♦ Communicating finite state machines (CFSM)—like StateChart, SDL, etc.
- ♦ Discrete event models like VHDL, Verilog, etc.
- ♦ Asynchronous message passing
- ♦ Synchronous message passing

In subsequent sections we will discuss about some of these important techniques.

8.2 STATECHART

StateCharts are among the widely used specification techniques, suitable particularly for control-dominated embedded system specifications. It is a modification of *finite state machine* (FSM) commonly used in controller specification. It may be noted that the normal FSM specification suffers from the following drawbacks:

1. Complexity of the state diagram increases dramatically as the number of possible states increases. This poses a major bottleneck in modelling large systems.
2. In traditional state machines, there is no concept of hierarchy. This, in turn, leads to the previous problem of state-explosion.
3. There is no support for concurrent constructs. Traditional state machine modelling is based on sequential transitions from one state to the next. Concurrent systems cannot be modelled in this manner, as various portions of the system may be in different states.

StateCharts introduced by *David Harel* in 1987, have some special features to overcome these problems. It allows the system to be simultaneously in more than one state. As a result, the concurrency can be modelled easily. The concept will become more clear when we discuss about the *superstates*. The states can communicate and synchronize between themselves. Some of the important features are as follows:

1. All orthogonal regions of the chart (that is, different subsystems within the whole system) accept events sent to the object.
2. One region may create an event as a result of a transition that is consumed by another orthogonal region.
3. A guard may be used to test if another region is in a certain state before allowing a transition to occur to the guarded state.

8.2.1 Modelling Hierarchy

System hierarchy can be modelled in StateCharts using the concept of *superstates*. A superstate contains other states within it. These are called substates of the superstate. The substates may again be superstates. A state which is not composed of any other state is called a *basic state*. It may be noted that a state may belong to more than one superstate.

There can be two types of superstates:

1. **OR-superstate:** This is commonly available in FSMs. The system, at any point of time, is in one of the constituent substates of an OR-superstate. For example, in the full FSM, it is always in one of its substates.
2. **AND-superstate:** An AND-superstate encompasses a number of substates, the system can be in all these states simultaneously. Though this apparently seems counter-intuitive, it can be utilized to model systems with different subsystems. For example, as shown in Fig. 8.1, the state S is an AND-superstate. It has two substates, each of which is an OR-superstate. Thus, the system can be simultaneously in one of the basic states $\{s_1, s_2, s_3, \dots\}$ for the first subsystem, and in one of $\{p_1, p_2, p_3, \dots\}$ for the second subsystem. If the number of states in the first subsystem is n_1 and that in the second subsystem be n_2 , then the total number of states needed in the AND-superstate is $n_1 + n_2$. However, in the absence of such a feature, a total of $n_1 \times n_2$ states will be needed to represent the system S . The AND-superstates are also said to consist of *orthogonal states*. Each of these orthogonal states can be used to model different regions of the system. For example, a telephone handset has at least two different functions—line monitoring and user interaction through buttons. The individual functions have their own state-transition behaviours. Thus, at any point of time, each of the subsystems is in one state. The overall handset can be considered simultaneously in two states. AND superstate helps in modelling this type of systems.

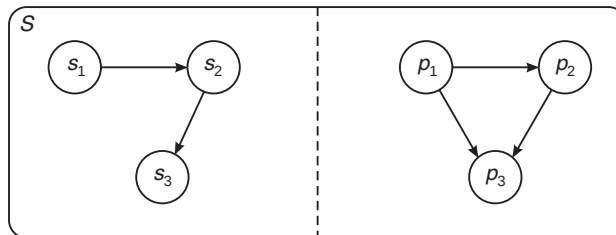


FIGURE 8.1 AND-superstate.

Next, we will highlight some of the features of *StateChart*.

1. **Transitions from superstates:** Figure 8.2 shows a hierarchical StateChart in which the state S is an OR-superstate. While the system is in state S , it is in either of the states P or Q . From state P , on occurrence of event C , it transits to state Q . In whatever substate the system be, on occurrence of event E , it will switch from state S to state T .
2. **Start and stop states:** Like FSM, StateChart may also have explicit start and stop states. Start states are indicated by simple large dots, whereas the stop states are encircled dots. For example, as shown in Figure 8.3, the overall StateChart has S as the start state. Within S , P is the start state. That is, whenever the state S is entered, unless otherwise stated, the state P

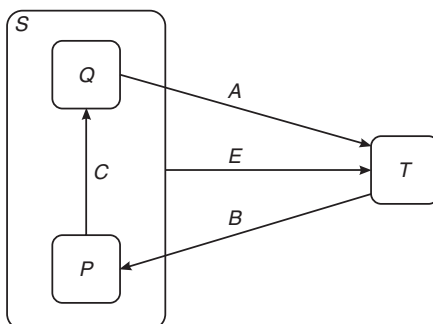


FIGURE 8.2 Transitions from superstate.

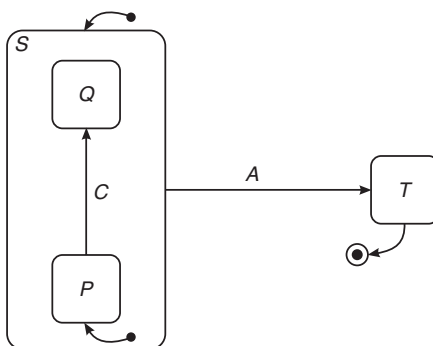


FIGURE 8.3 Start and stop states.

3. *History mechanism:* Sometimes, it may be required that the system when transiting into a superstate, should enter into the substate it was, when the state was visited last time. This can be accomplished by the history mechanism (as shown in Figure 8.4). Here, while transiting out of state *S*, the system remembers the substate in which it was. Later on, while transiting from *T* to *S*, the arrow reaching *H* indicates that it should restart from the state the system remembered.

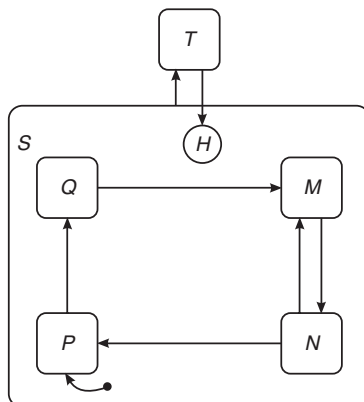


FIGURE 8.4 History mechanism.

4. *Timers*: These are very important for real-time embedded systems. A timer is also represented as a state with a time value. Once the state has been entered, if no event occurs to create a transition out of the state, the timer will time out and the transition marked *timeout* from this state will take place. This has been shown in Figure 8.5, in which the state *P* is a timer with a timeout period of 10 ms. If no other transition from the state takes place within this 10 ms time, the transition labeled *timeout* to state *S* takes place.

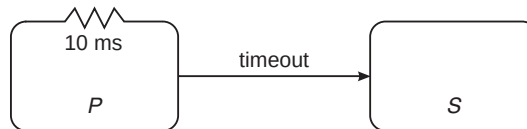


FIGURE 8.5 Timer state.

5. *Edge labels*: The transitions in a StateChart are labeled as noted in the following.

`< event > [< condition > ]/[ < reaction > ]`

Thus, there must be an *event* associated with a transition. The *< condition >* acts as a Boolean guard. It is an expression involving values of variables. The *condition* is enclosed within '['and']'. The *< reaction >* part sets some variables to new values. Thus, some typical edge labels can be,

key-pressed / *on* := 1

key-pressed [*on* = 1]/ *on* := 0

key-pressed [*on* = 0]/ *on* := 1; *light* = 1

6. *StateChart simulation*: StateChart execution consists of three phases. In the first phase, the events and conditions are evaluated from the current state of the system. The transitions that are enabled by the process are identified. In the second phase, for the enabled transitions, the *reactions* are evaluated, but are not immediately assigned to the variables, rather, they are kept in temporaries. In the third and final step, the transition to the new state takes place and the variables are assigned the values evaluated in the temporaries. This three-step semantics helps in simulating situations in which a variable is being modified in a transition, whereas, in another simultaneous transition, the variable appears in the *condition* part.

It may be noted that the visibility of events is limited to the step following the one in which they are generated. The values do not live forever. On the other hand, a variable, once assigned a value, will remember it till a new assignment overwrites it.

EXAMPLE 8.1

Digital watch: Next, we look into the StateChart specification of a digital watch. It has three different modes—*normal display* (default), *update* to update time and date settings, *alarm-set* to set alarm. In *normal display* mode, normally the time is displayed. However, on pressing button *a*, it switches to display date. On further pressing of *a*, it comes back to display time again. From normal display mode, on pressing button *c*, it switches to the mode to *update* time and date settings. Similarly, on button *b*, it switches from normal display mode to *alarm-set* mode, in which, it can set alarm time and chime. This has been shown in Figure 8.6. Apart from this main function, the clock has an alarm mode to ring alarms. There is a background light that can be turned on/off anytime. This has been shown in Figure 8.7.

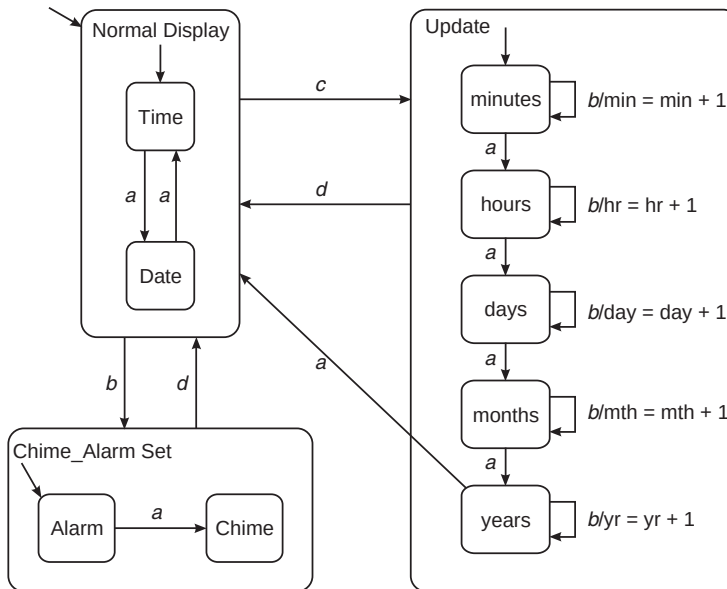


FIGURE 8.6 StateChart for modes of digital watch.

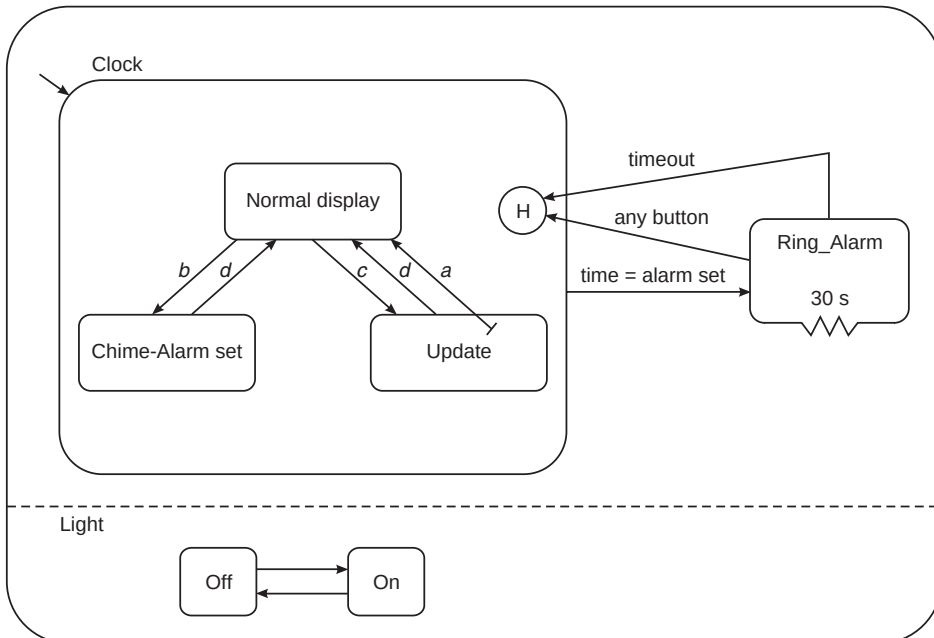


FIGURE 8.7 Concurrency in digital watch.

EXAMPLE 8.2

Date entry mechanism: Let us assume a system to be consisting of a numeric key-pad (keys 0 to 9) and the following special keys to enter a valid date.

1. A *delete* key to delete the character entered last.
2. A *delimiter* key to give delimiter between the day, month, and year parts.
3. An *enter* key to complete the data entry.

The corresponding StateChart has been shown in Figure 8.8. The diagram is self-explanatory.

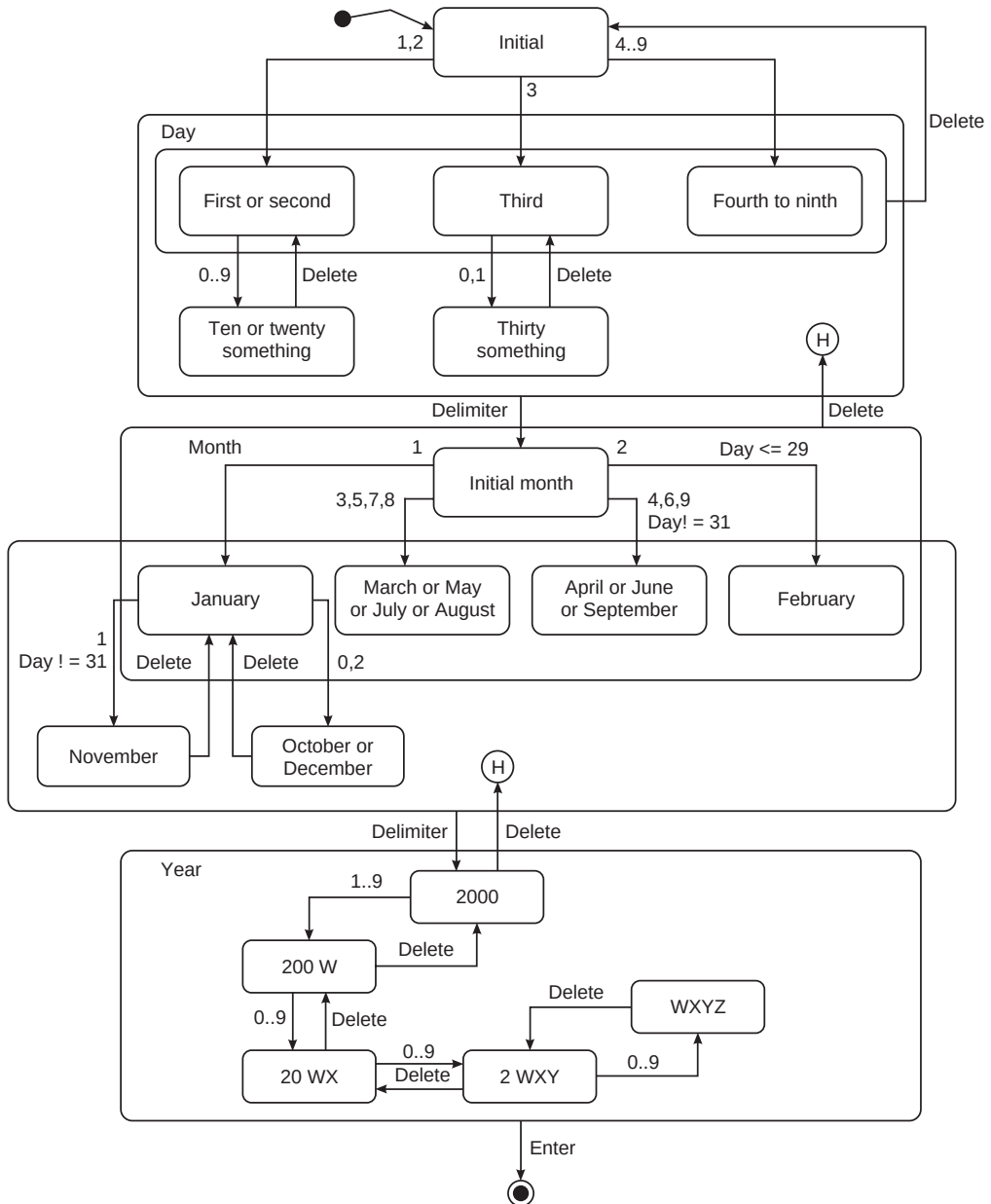


FIGURE 8.8 StateChart for date entry.

Disadvantages of StateChart

The following are some of the short comings identified for the StateCharts.

1. The concept of AND-superstates works well for simulation. However, if implemented in software, there hardly exists any efficient realization of those, particularly for small processors unable to support concurrent processes as in UNIX. Hardware being inherently parallel in nature, it follows the AND superstate behaviour more closely.
2. There are no object-oriented features. Thus, the object-oriented design methodologies cannot be practiced.
3. The events and variables are known to all the states in a StateChart. This requires the ability to broadcast any change in their values to the entire StateChart. This may be possible for centralized systems, but becomes a difficulty for modelling distributed systems. It may be noted that distributed systems normally work through a message-passing mechanism, in which messages are often directed to a single or a group of processes. Thus, the occurrence of an event or changing the value of a variable will create a large number of messages in the system.
4. StateCharts do not have any programming constructs. This makes it unsuitable for complex computation. It is also not possible to describe hardware structures and non-functional behaviour.
5. Thus, StateCharts are particularly suitable for centralized, control-dominated system specification. There exists a number of commercial tools like *StateMate*, *StateFlow*, *BetterState* etc. built around StateChart. Many of these tools can automatically synthesize the corresponding C/VHDL code from the specification.

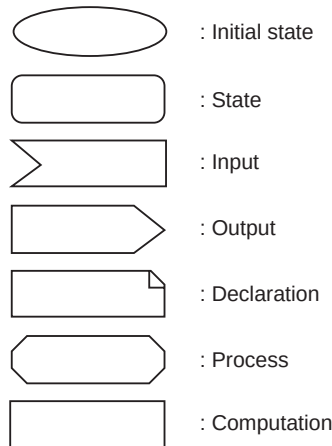
8.3 SPECIFICATION AND DESCRIPTION LANGUAGE (SDL)

SDL is a premier language for specification, design, and development of real-time systems, in particular, the telecommunication applications. The currently available version is *SDL-2000*, proposed in 1999 by ITU-T and some modifications later on. It is a graphical language and is based on the concept of *Extended Finite State Machine* (EFSM). An *SDL* system consists of one or more communicating *agents*, with the outermost agent communicating with the environment. An agent may in turn contain other agents in a hierarchy, definition of behaviour by EFSM, data variables of value or references data types and communication, based on asynchronous message exchange.

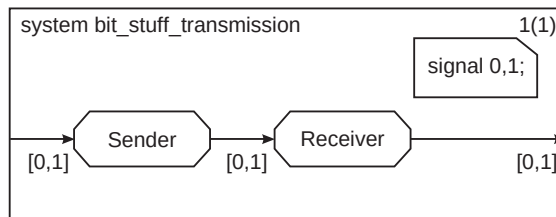
SDL specification consists of a set of diagrams. The top-level diagram is the outermost agent, it shows the connection between the system components and the environment. Each diagram has one or more pages, each page having the following:

- ♦ a *frame* with some information attached to the outside.
- ♦ the *diagram heading* showing the kind and the identity of the item described, in the top left corner.
- ♦ *page name* and *number of pages* in the top right corner.

The symbols normally used in a diagram are as shown in Figure 8.9. Figure 8.10 shows a simple system-level diagram. This is a bit-stuffing system that does not allow the occurrence of bit-string “011110”. For doing this, as soon as the *sender* finds a sequence “01111”, it forcefully stuffs a ‘1’ bit. In the *receiver*, this extra stuffed ‘1’ bit is removed. The diagram consists of two

**FIGURE 8.9** Symbols in SDL.

processes, *sender* and *receiver*. A *channel* ($\rightarrow$) shows the signals between two agents, or between an agent and the environment of a diagram. The signal names are listed near the arrow-head between '[' and ']' symbols. The arrow-head shows the direction. Channels may also be named. Name of channels, signals, etc. can have letters, digits and underscore characters. For example in Figure 8.10, the signals are named 0 and 1.

**FIGURE 8.10** SDL system for bit-stuffed transmission.

A system diagram can contain *process* agents or *block* agents. Instances within a block agent work concurrently and asynchronously, whereas instances within a process are scheduled one at a time. A block agent can contain within it other block agents or processes, whereas, a process can contain other process agents only within it. The boundary of agent diagram also serves the purpose of information hiding. Items defined within enclosing agent diagram are visible in inner diagrams, but not the outer way. This is different from *StateChart* that follows a broadcast mechanism.

An agent containing one state machine can have a behaviour graph as its diagram. Figure 8.11 shows the *sender* process containing a finite state machine. It has a *start* state and other states, like *initial*, 0, 01, 0111, 011111 along with associated inputs and outputs. From the initial state onwards, based upon the input bits, it traverses through the states. For example, from the state *initial*, upon getting an input 0, it outputs a bit 0 and goes to state 0. While, on getting 1, it outputs 1 and loops back to the state *initial*. Finally, from state 0111, upon getting another 1 as input, it outputs two 1's and goes to a state named 011111. Thus, it pads one extra 1 to avoid the pattern "011110" in the output.

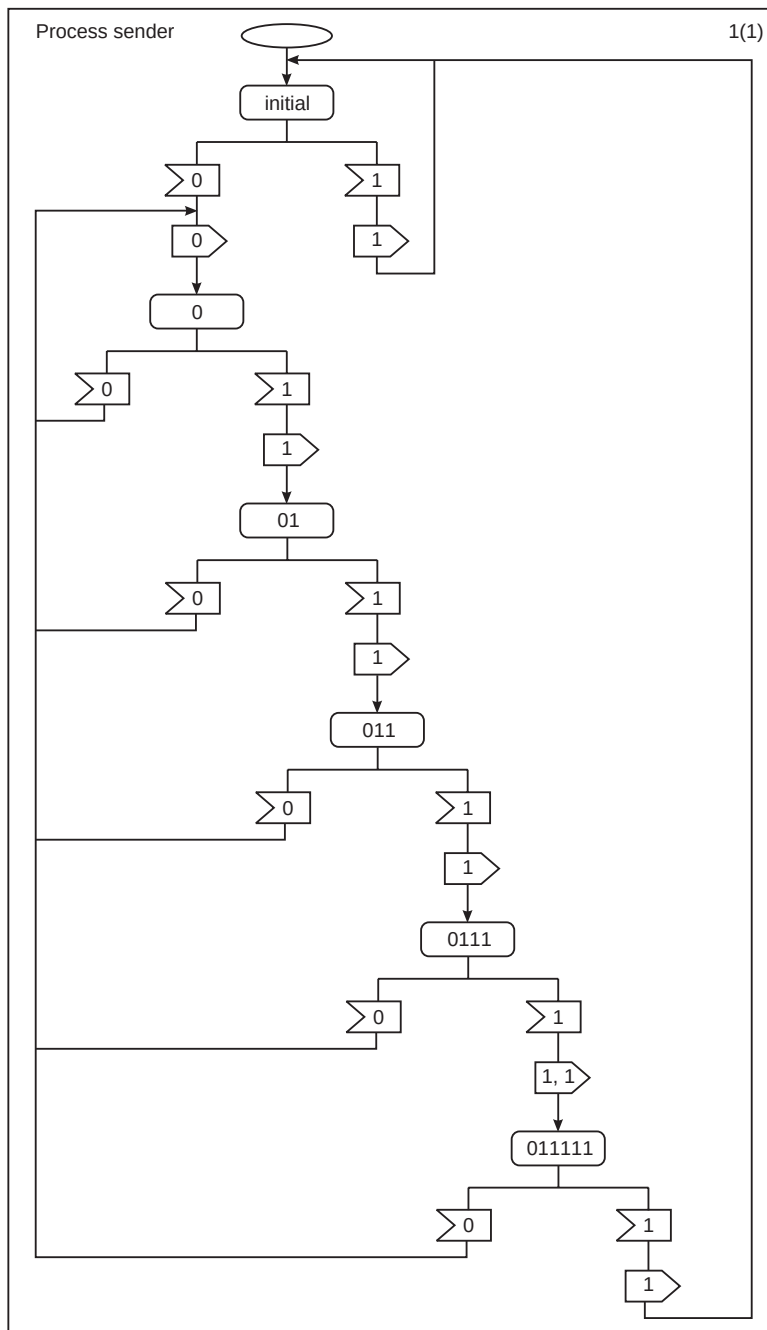


FIGURE 8.11 Sender process in bit-stuffed transmission.

SDL extends the concept of FSM in the following two ways.

1. Each agent has got a queue associated with it that stores all the signals received in a first-in-first-out manner.

2. Data can be received in signals, stored in variables, used in expressions, used to decide the behaviour of the agent, and passed out in output signals.

The FSM waits in its current state until one of the signals that can be consumed in that state is available.

The *receiver* process has been shown in Figure 8.12. After initiation, it waits in state *wait* and scans the queue for a 0 or 1. Two variables *count0* and *count1* have been declared and initialized in the data declaration *dcl*. It uses the symbols *decision* (for example, checking *count0*, *count1* etc.), *tasks* (such as, computations involving *count0*, *count1*), *text* (the *dcl*), and *stop* (×). The stop condition is not reachable if the code behaves correctly.

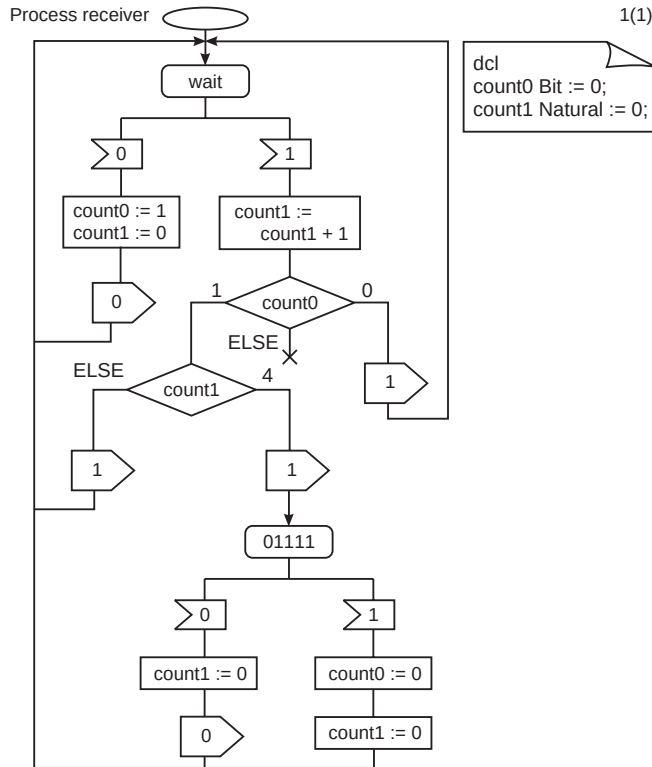


FIGURE 8.12 Receiver process in bit-stuffed transmission.

8.3.1 Signal Communication

Signals are transmitted via *channels* from an output agent to an input agent. Signals are declared just like variables. The channels may be named. For example, consider a process interaction diagram as shown in Figure 8.13. Here, block *B1* has two processes *P1* and *P2*. The signals *A* and *B* can be sent over the channels *Sw1*, *Sw2*, or *Sw3*. Now, in the behaviour of the processes, the signal transmissions can be specified in different ways as follows:

1. Through process identifiers using *Pid* expressions. Four *Pid* expressions are available to each agent for communication:

- ♦ *self*—agent's own identity
 - ♦ *parent*—agent that created this agent, *null* for initial agents
 - ♦ *offspring*—the most recent agent created by this agent
 - ♦ *sender*—the agent that sent the last signal input
2. Through explicit channel names, such as *Sw1*, *Sw2*, *Sw3*, etc.
 3. Implicitly, by the fact that the signal name identifies the channel as well. For example, in Figure 8.13, signal *B* is always sent via channel *Sw1*.

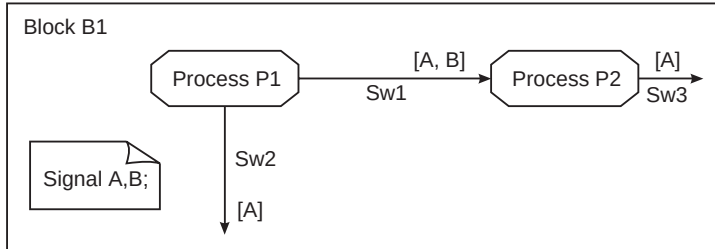


FIGURE 8.13 Process interaction diagram.

Figure 8.14 shows a few examples about how the signals can be specified for transmission. It may be noted that an agent can create instances of other agents. There can exist multiple instances of an agent, particularly when realized in software. Thus, the definition of an agent includes how many instances of agents will be there, and the maximum number of instances. The default is one initial instance, and no limit on the maximum. For example, in Figure 8.15(a), process *P1* is defined to have 1 initial instance and a maximum of 1 instance, whereas process *P2* has 1 initial instance and at the most 3 instances. Agents can be created by other agents in a create request (shown in Figure 8.15(b)) as part of a transition.

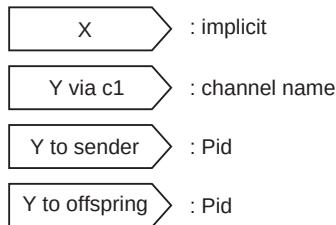


FIGURE 8.14 Signal transmission specification.

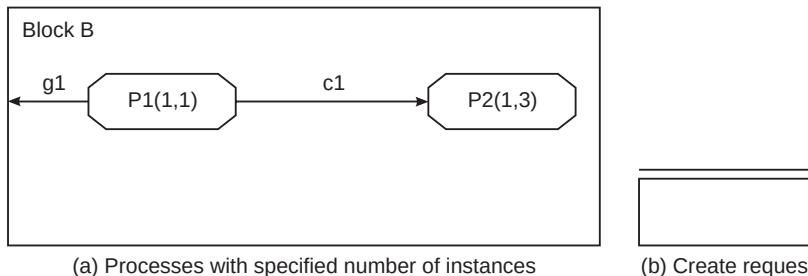


FIGURE 8.15

8.3.2 Timer

An agent may have defined *timers* within it. A timer can be created by a definition like,

timer *t4* := 10.5;

A timer can be started with a *set* and can be cancelled with a *reset*. For example,

set(now + 3.2, <i>t4</i>);	– sets the timer <i>t4</i> to 3.2 from current time
set(<i>t4</i>);	– sets the timer <i>t4</i> to the duration given in the timer definition from current time
reset(<i>t4</i>);	– resets the timer <i>t4</i> before it expires
active(<i>t4</i>);	– tests if the timer <i>t4</i> is active

It may be noted that timer definitions are not allowed in state/procedure diagrams. A typical use of timer has been shown in Figure 8.16.

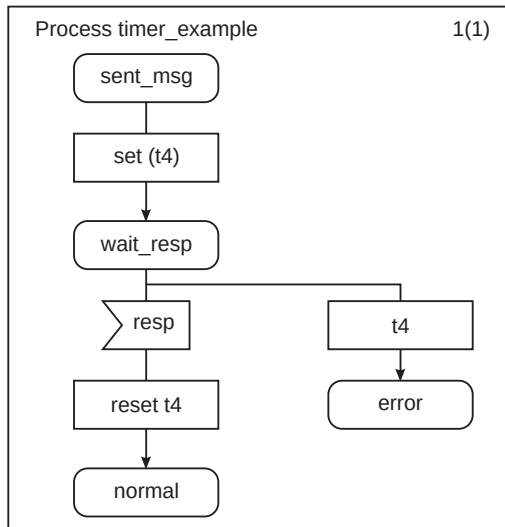


FIGURE 8.16 Timer example.

SDL-2000 contains many other important features, such as procedures, composite states, data types, exception handling, and so on. A detailed discussion on these is beyond the scope of this book. One should refer to the website <http://www.sdl-forum.org> for complete documentation of SDL and find the recent developments.

8.4 PETRI NETS

Petri Nets (also called *Place/Transition Net* or *P/T Net*) is one of the several mathematical representations of discrete distributed systems that have been widely used for system modelling in many fields of science. It was invented by *Carl Adam Petri* in 1962 as part of his Ph.D. thesis.

Two important intrinsic features of Petri Nets are *concurrency* and *asynchronous* nature. These, together with generality and flexibility have stimulated their applications in several areas including specification and verification of real-time embedded systems. We will first look into the basic Petri Net structure. However, this cannot be directly used to model embedded

systems as it does not have the notion of time. The extensions to this basic structure will be discussed next to show the modelling of embedded systems using Petri Nets.

8.4.1 Basic Petri Nets

A *Petri Net* consists of *places*, *transitions* and *directed arcs*. Arcs can run between places and transitions. These can never be between places, or between transitions alone. Graphically, the places are represented by circles and transitions by solid lines. The places from which an arc runs to a transition are called *input places* for that transition. Similarly, the places to which arcs run from a transition are called *output places* for that transition. Each place may contain any number of *tokens*. In basic *Petri Net* structure, a token is represented by a small filled circle. The distribution of tokens among the places, at any point of time, is called a *marking*. A transition is *enabled* when all its input places have sufficient number of tokens. An enabled transition may *fire*, consuming the tokens from its input places, and placing a specified number of tokens at its output places. The entire firing operation is *atomic*, that is, once a transition fires, all the subtasks, as listed above, are performed together. It may further be noted that the execution of *Petri Nets* is non-deterministic. That is, given a number of enabled transitions at a time, zero or any one of them can fire. It is not necessary for an enabled transition to fire till the infinite time. Due to this non-determinism, *Petri Nets* are suitable for modelling concurrent behaviour of distributed systems. Figure 8.17 shows a very simple *Petri Net* consisting of two places $p1$ and $p2$, and a transition $t1$. Initially, only the place $p1$ has a token. Thus, the transition $t1$ is enabled. Once the transition fires, the token is removed from the input place $p1$ of $t1$ and is put in the output place $p2$.

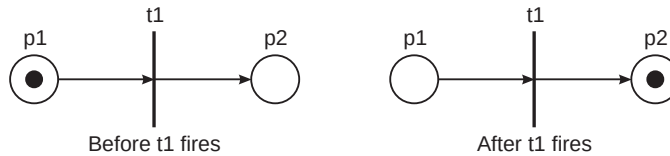


FIGURE 8.17 A simple Petri Net.

Formally, a *Petri Net* is a 6-tuple (S, T, F, M_0, W, K) , where,

- ♦ S is a set of places.
- ♦ T is a set of transitions.
- ♦ F is a set of arcs with the restriction that it can run between places and transitions only, that is $F \subseteq (S \times T) \cup (T \times S)$.
- ♦ $M_0 : S \rightarrow N$ (N is the set of natural numbers) is the initial marking, for each place $s \in S$, there are $n \in N$ tokens.
- ♦ $W : F \rightarrow N^+$ is the set of arc weights. It assigns to each arc $f \in F$ some $n \in N^+$ denoting how many tokens are consumed from an input place by a transition. For the output arcs, the weight indicates the number of tokens to be deposited at the output place.
- ♦ $K : S \rightarrow N^+$ is a set of capacity restrictions. It assigns a positive integer $n \in N^+$ to each place $s \in S$ denoting the maximum number of tokens that the place can hold.

A weighted *Petri Net* has been shown in Figure 8.18 before and after the firing of transition $t1$. It may be noted that the weights for edges with no label are taken as unity. Next, we will look into a few examples of system modelling using basic *Petri Nets*.

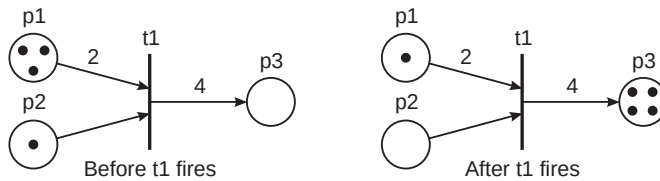


FIGURE 8.18 A weighted Petri Net.

EXAMPLE 8.3

Modelling of a system on an assembly line that counts 5 items and then sends a signal to the operator has been shown in Figure 8.19. It has four places $p1$, $p2$, $p3$, $p4$ and three transitions $t1$, $t2$, $t3$. Their meanings are as follows:

- $p1$: item produced on assembly line
- $p2$: count items
- $p3$: signal is on
- $p4$: signal is off
- $t1$: sensor recognizes item going by
- $t2$: turn signal on
- $t3$: turn signal off

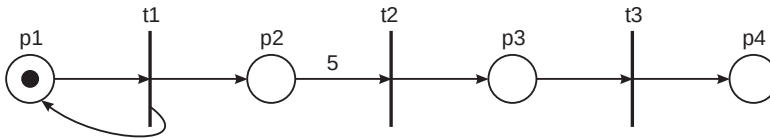


FIGURE 8.19 Petri Net of Example 8.3.

EXAMPLE 8.4

Mutual exclusion: Consider a road-crossing in which two roads (1) and (2) are crossing each other [see Figure 8.20(a)]. To avoid collision, at any point of time, traffic can be allowed in only

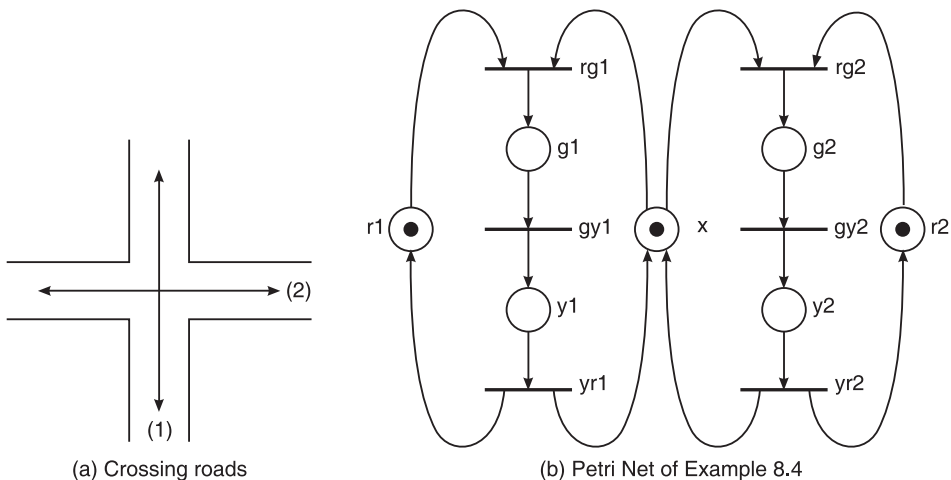


FIGURE 8.20

one road. Accordingly, signals have been installed, consisting of three lights—Red (r), Yellow (y), and Green (g). To model this system using *Petri Net*, consider the diagram shown in Figure 8.20(b). The places $r1$, $g1$ and $y1$ represent places corresponding to red, green, and yellow lights being on for road (1). Similarly, $r2$, $g2$, and $y2$ represent the same for road (2). The place x stands for exclusive access permission. The transition $rg1$ can fire, provided the places $r1$ and x have tokens. The rest of the diagram can be followed easily. The places $g1$ and $g2$ cannot get tokens simultaneously, ensuring that traffic is allowed only in road (1), only in road (2), or in none at any point of time.

EXAMPLE 8.5

Readers–Writers synchronization: This is another classical problem on synchronization, in which there are k number of tasks accessing a shared resource. Out of these k tasks, some are *readers* that want to read the shared item, but do not want to modify it. The remaining are *writer* tasks that modify the shared item. Naturally, there is no restriction on the number of reader tasks proceeding simultaneously, however, only a single writer can access the item. When a writer task is accessing the shared item, no other reader or writer task can be allowed to access it. The Petri Net corresponding to this has been shown in Figure 8.21. It may be noted that for a writer to be allowed, it consumes one token from the place *Processes* and k tokens from the *Repository*. On the other hand, for a reader to be allowed, it needs one token each from *Processes* and *Repository*. On finishing, a writer replenishes one token to *Processes* and k to the *Repository*, while a reader replenishes one token to each place.

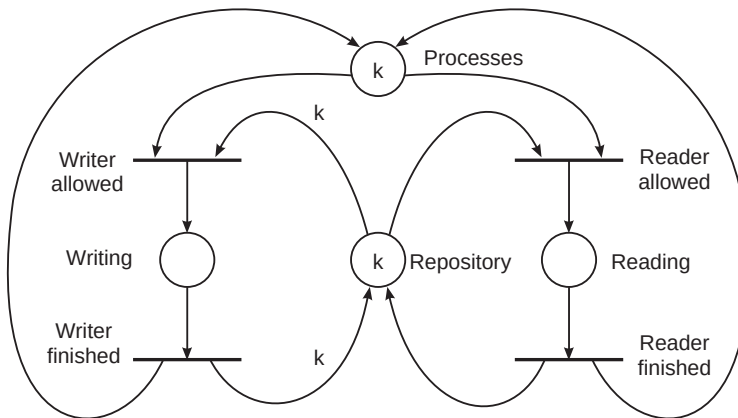


FIGURE 8.21 Readers–Writers synchronization of Example 8.5.

EXAMPLE 8.6

Communication protocol: In this example, there are two processes—*Process1* and *Process2*. *Process1* sends messages to *Process2* and *Process2* acknowledges. After sending a message, *Process1* waits for an acknowledgement from *Process2*. The Petri Net model of the protocol has been shown in Figure 8.22.

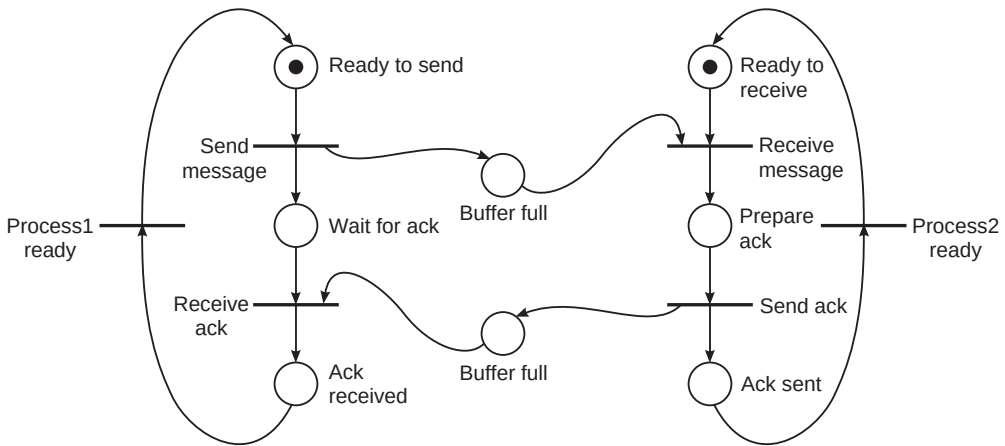


FIGURE 8.22 Communication protocol of Example 8.6.

EXAMPLE 8.7

A vending machine: This example considers a chocolate vending machine that sells two types of chocolates worth ₹15 and ₹20. It takes as input two types of coins, one worth ₹5 and the other worth ₹10. There are two more buttons to input the type of chocolate asked for. After entering the requisite amount of money, the user inputs the type and the chocolate is dispensed. The Petri Net model for the system has been shown in Figure 8.23.

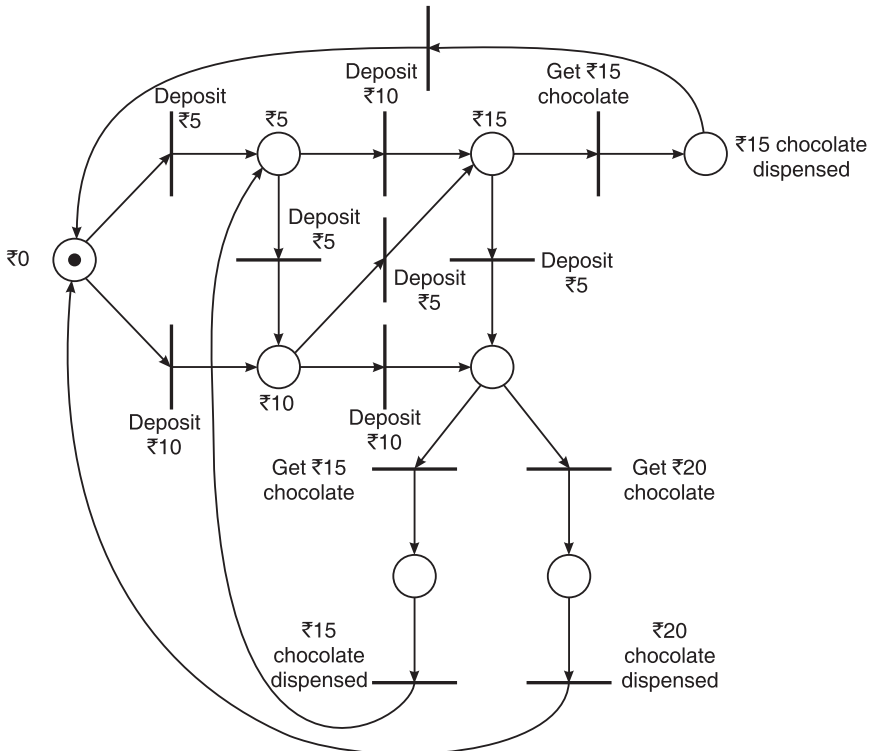


FIGURE 8.23

8.4.2 Properties of a Petri Net

On a system modelled using Petri Net, we can verify several properties that make the modelling interesting. The following are some of the important properties. The process allows us to verify many physical properties of the system.

1. **Termination:** Whether the net terminates.
2. **Immediate reachability:** Is a state reachable when a transition fires?
3. **Reachability:** Is a state eventually reachable?
4. **Liveness** in all states: Is there one transition that can fire?
5. **Partial deadlock:** Is there a state in which there is at least one transition that can never fire?
6. **Deadlock:** Is there a state in which none of the transitions can fire?
7. **Safety** in all states: Does each place contain at most one token?
8. **Boundedness** in all states: Is there a limit to the number of tokens that can be in one place?
9. **Conservativeness:** Is the total number of tokens in the Petri Net constant?

In the following, we discuss about some of these important properties.

1. **Reachability:** The state of a Petri Net is identified via its *marking*. As noted earlier, marking corresponds to a distribution of tokens amongst the places of the net. A reachability analysis answers the question, whether a given marking (state) is reachable from another specified marking (state). For example, Figure 8.24 shows two markings M and M' of a net. It can be argued that M' is not reachable from M .

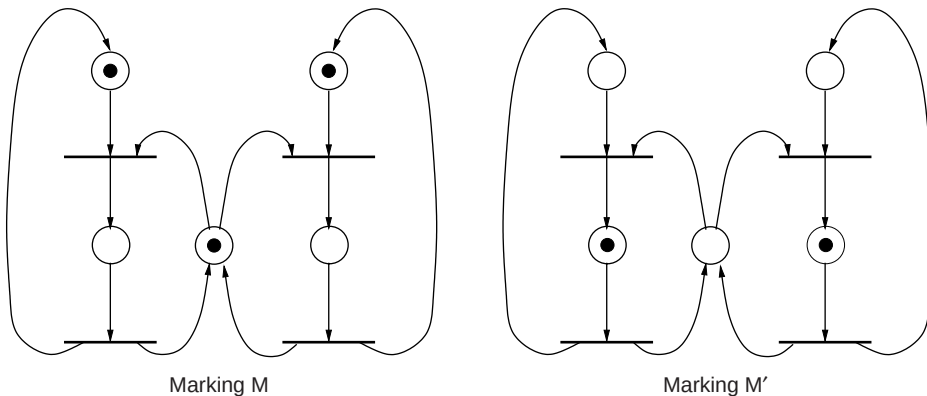


FIGURE 8.24 Reachability example.

2. **Liveness:** A transition T is said to be live, if for every marking M_1 reachable from initial marking M there exists a marking M_2 that enables T and is reachable from M_1 . The total net is live if all its transitions are live. Figure 8.25(a) shows a net which is not live in the sense that if transition T_1 fires, there is no chance of T_2 to fire in future. However, the net shown in Figure 8.25(b) is live.

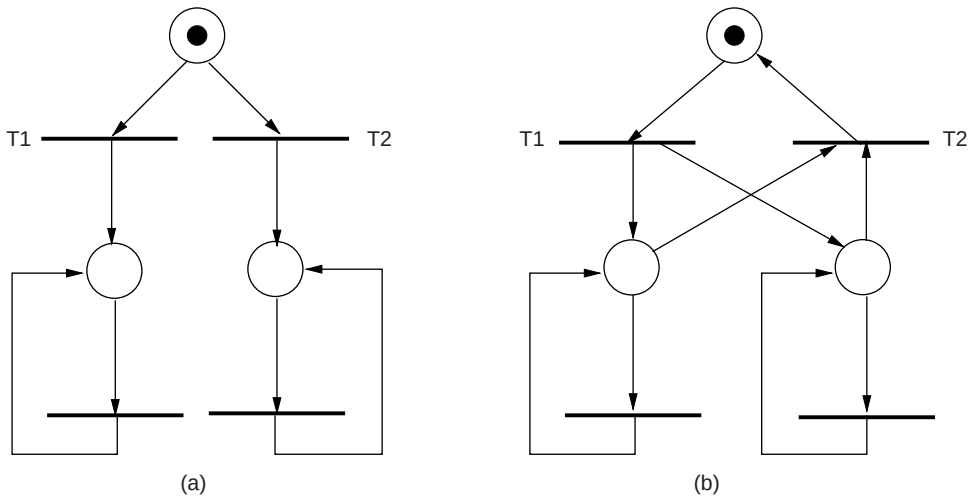


FIGURE 8.25 Liveness example.

3. **Boundedness:** It refers to the maximum number of tokens that each place of a net can contain. For the net shown in Figure 8.26(a), the place P_1 can hold any number of tokens. Each time the transitions on the left of P_1 fire, one token is deposited into P_1 . Thus, the net is not bounded. However, for the net shown in Figure 8.26(b), the places P_1 and P_2 each can hold at most two tokens, whereas the other places can hold just one token. Thus, the net is bounded. A net is said to be *safe* if its bound is 1. Figure 8.26(b) also shows the property of *place invariants*. The places P_1 and P_2 always satisfy the condition that the sum of the number of tokens in them is always equal to two.

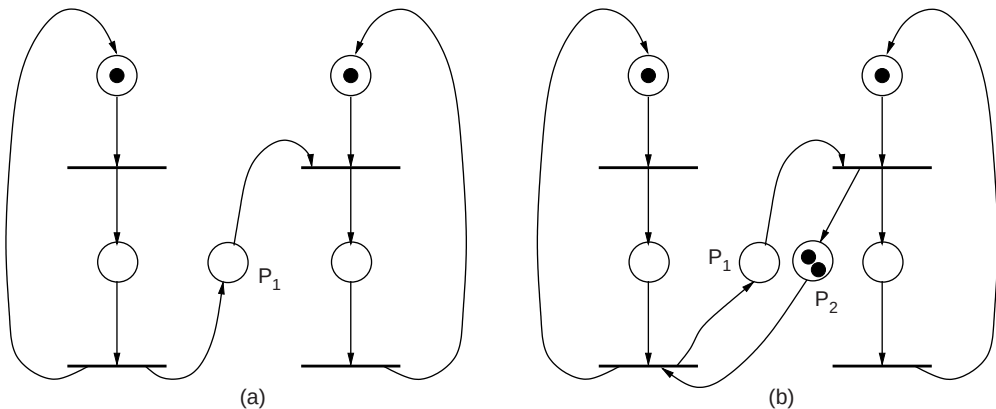


FIGURE 8.26 Boundedness and place invariants example.

8.4.3 Extensions to Petri Nets

The following are the major shortcomings of conventional Petri Nets discussed so far:

1. It lacks hierarchy in the specification. Thus, even for a moderately complex system, the net becomes too big, due to state-explosion.

2. The tokens represented as filled circles lack expressiveness. No value is transferred in the process. This makes the model limited.
3. The conventional Petri Nets lack the notion of time. However, particularly for modelling embedded applications, time is a very crucial factor.

Several extensions have been proposed in the literature to overcome these drawbacks. The following presents some of these, before going to a full-fledged discussion on Petri Net based models of embedded systems.

1. **Time extended Petri Nets:** There are four major approaches proposed so far:
 - ◆ *Timed Petri Nets:* An execution time is attached with each transition to associate a finite duration for firing. Here, transitions are not instantaneous. A transition has to be fired as soon as it is enabled. Tokens are immediately removed from its input places. After a time delay, tokens are deposited in the output places. Thus the state of the system is not always clearly represented during the process.
 - ◆ *Time Petri Nets:* Here, two values of time are associated with each transition starting from the moment the transition is enabled. One is the minimum time value and the other is the maximum time value. Between these two times, the transition has to fire, unless it is disabled by the firing of another transition.
 - ◆ *Timed Place Transition Nets:* Time information is associated with places, rather than transitions. This means, each place has got some associated delay. A token must remain in that place for this time interval, before being removed by a transition.
 - ◆ *Tokens holding time information:* A token has got a time-stamp about when it was created. We will discuss this scheme in detail in the following sections.
2. **Coloured Petri Nets:** These were introduced in late 1970s and have a very strong mathematical theory developed around them. Here tokens often represent objects (such as resources, goods, humans, etc.) in the modelled system. The tokens are thus *coloured*, that is, *typed*. The value of the token is called its colour. Transitions use the values of the consumed tokens to determine the values of the produced tokens. Thus, a transition describes the relation between the values of the input tokens and the values of the output tokens. The transitions may also have some preconditions that take colour of token to be consumed into consideration. Thus, coloured Petri Nets can also be used to model time through token values.

8.4.4 Embedded System Modelling with Petri Nets

There are many works reported to model embedded systems using extended Petri Nets. In this direction, we will look into two important techniques to specify embedded systems. These are:

1. *PRES*, Petri Net-based Representation for Embedded Systems
2. Dual transitions Petri Nets

PRES

PRES is an extension to the classical Petri Nets. It captures explicitly the timing information, allows a hierarchical representation of a system. Tokens are allowed to carry information. *PRES* can also express concurrency and sequential behaviour. A formal description of *PRES* is as follows:

PRES is a five-tuple (P, T, I, O, M_0) , where,

$P = \{p_1, p_2, \dots, p_m\}$ is a finite non-empty set of places.

$T = \{t_1, t_2, \dots, t_n\}$ is a finite non-empty set of transitions.

$I \subseteq P \times T$ is a finite non-empty set of input arcs defining the flow relation between places and transitions.

$O \subseteq T \times P$ is a finite non-empty set of output arcs defining the flow relation between transitions and places.

M_0 is the set of initial marking.

The tokens are also modified. A token is defined to be a pair $k = (v_k, r_k)$, where v_k is the token value and r_k is the token time. The token time is a positive real number representing the time-stamp of the token. For any place, the type of token that it can hold is identified by a *type function* $\tau : P \rightarrow K$. The *pre-set* of a transition t , denoted as ${}^o t = \{p \in P \mid (p, t) \in I\}$ is the set of input places of t . Similarly, the *post-set* of a transition t , denoted as $t^\circ = \{p \in P \mid (t, p) \in O\}$ is the set of output places of t . A transition has got associated with it, the following:

1. *Output functions*: For every place in t° of a transition t , there exists an output function mapping the token values at places of ${}^o t$ to the token value at the output place. That is,

$$\forall p_j \in t^\circ, \exists f_j : \tau(q_1) \times \tau(q_2) \times \dots \times \tau(q_a) \rightarrow \tau(p_j)$$

with ${}^o t = \{q_1, q_2, \dots, q_a\}$ and $t^\circ = \{p_1, p_2, \dots, p_b\}$.

2. *Function delays*: For every function associated with a transition, there is a delay.
3. *Guard*: A guard G_t of a transition t is a Boolean condition that must hold true when all its input places hold token, for the transition to be enabled.

A transition is *enabled* if all its preset places are *marked* (that is, contain tokens), output places are empty (excepting those places acting both as input and output), and its guard is asserted. Every enabled transition has got an associated *trigger time* that represents the time instant at which the transition may fire. The trigger time of an enabled transition is the maximum *token time* of the tokens at its preset places. When a transition fires, each of its output places get a token. The *token time* of a token in an output place is equal to the sum of the trigger time of the transition and the delay of the associated output function.

Representing the hierarchy: A transition t and its surrounding places can be substituted by a single Petri Net structure keeping the functionality unaltered. Similarly, a Petri Net structure can be represented by a single transition. This way, a complex structure can be represented in a hierarchical fashion using Petri Net model.

EXAMPLE 8.8

Figure 8.27(a) shows the PRES representation of the following multiplication algorithm.

```
int mult( int a, int b)
{
    int x, y, z;
    x = a;
    y = b;
    z = 0;
    while (y > 0){
        z = z + x;
        y = y - 1;
    }
    return z;
}
c = mult(a, b);
```

The model consists of four transitions t_1 , t_2 , t_3 , and t_4 . There are six places— A , B , C , X , Y , Z . The arcs are labeled with the function they compute. Thus, the label a from the place A to the transition t_1 indicates that the value of a will come as token value to the transition. All output functions of a transition are assumed to have same delay, marked within the transition. Thus, each of the output functions associated with transition t_3 has a delay of 6 units and so on. Some transitions, like t_3 and t_4 are guarded. Transition T_3 can fire provided its input places X , Y and Z have tokens and $y > 0$. Once it fires, it computes $z = z + x$ and $y = y - 1$ with delay 6. When y becomes equal to zero, the guard condition of t_4 is true. It can fire, provided its input places X , Y and Z have tokens. In that case, after a delay of 4 unit, C gets the value of z . It may be noted that t_4 does not need the value of X , however, the place X must have a token at that time for the consistency of operation. The entire multiplication can be represented hierarchically by a single transition as shown in Figure 8.27(b).

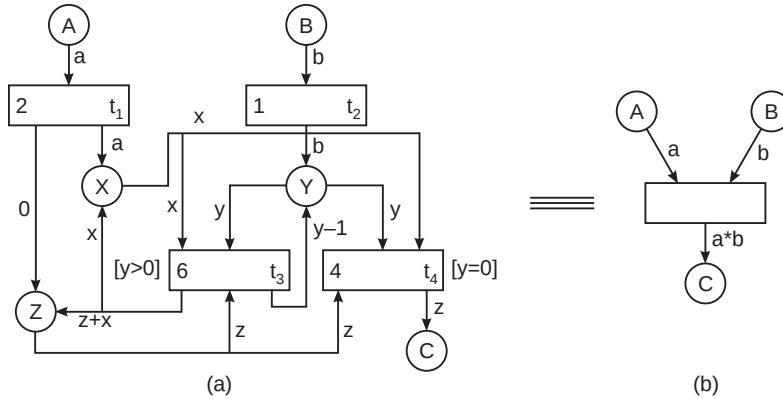


FIGURE 8.27 PRES structure of multiplication algorithm.

Dual transitions Petri Nets

The concept of dual transitions Petri Nets originated from the fact that an embedded system specification usually consists of both control functions and data operations. The extensions proposed here allow the representation of two mutually exclusive domains—control and data. The structure is a seven-tuple, $N = (P, T, Q, F_C, F_D, W, G)$, where,

$P = \{p_1, p_2, \dots, p_n\}$ is a finite set of places.

$T = \{t_1, t_2, \dots, t_m\}$ is a finite set of control transitions.

$Q = \{q_1, q_2, \dots, q_h\}$ is a finite set of data transitions.

$F_C \subseteq (P \times T) \cup (T \times P) \cup (P \times Q)$ is a set of arcs describing the control flow relation.

$F_D \subseteq (P \times Q) \cup (Q \times P) \cup (P \times T)$ is a set of arcs describing the data flow relation.

$W : \{F_C \cup F_D\} \rightarrow Z$ is a weight function, Z being the set of positive and negative integers including zero.

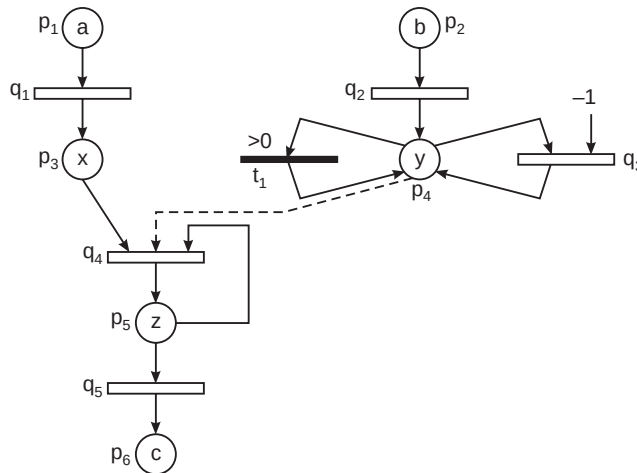
$G : P \rightarrow \{0, 1\}$ is the guard function.

The graphical representation of the structure consists of circles representing places, solid bars representing control transitions, rectangles representing data transitions. Arcs represent either control-flow relation (dashed directed arcs) or data flow relation (directed arcs). The concept of *pre-set* and *post-set* remain unaltered. However, as shown in Table 8.1, for control and data transitions, the sets are represented differently.

TABLE 8.1 Symbols of pre- and post-sets

Domain	pre-set	post-set
Control	$\cdot x$	$x \cdot$
Data	$^{\circ}x$	x°

The multiplication of Example 8.8 has been shown in Figure 8.28. To perform multiplication, first, the values are placed at places p_1 and p_2 . Thus, data transitions q_1 and q_2 are enabled. Firing them leads to movement of the two values to the places p_3 and p_4 . Some operations mapped onto data transitions require a control signal to be activated. For example, transition q_4 is enabled when p_4 holds a token.

**FIGURE 8.28** Dual transitions Petri Net structure of multiplier algorithm.

8.5 UNIFIED MODELLING LANGUAGE (UML)

The *Unified Modelling Language* (UML) was originally proposed for modelling complex, software intensive systems. It provides quite a few advantages over *SDL*. It is less formal, and thus can be used at an early stage to structure and analyse the concepts of an application domain before the functional design is made. *SDL* does not support relations in object modelling. Thus, *UML* can be used to develop the formal functional design in *SDL* using the object models of UML.

UML is a graphical language consisting of different types of diagrams that can be used to describe a model from different angles. The diagrams can be classified into three categories:

- ◆ *Behaviour diagrams*—describing behavioural features of a system. This includes *activity*, *state machine*, *use case*, as well as the following four interaction diagrams.
- ◆ *Interaction diagrams*—a subset of behaviour diagrams emphasizing object interactions. This includes *communication*, *interaction*, *overview*, *sequence*, and *timing* diagrams.

- ◆ *Structure diagrams*—depict the elements of a specification that are irrespective of time. The constituents are *class*, *composite structure*, *component*, *deployment*, *object*, and *package* diagrams.

In the following subsections, we briefly describe each of these diagrams.

8.5.1 Activity Diagram

This represents the operational step-by-step workflows of components in a system. It gives the overall flow of control. UML activity diagrams may be considered as object-oriented equivalent of *flow charts* and *data flow diagrams*. Thus, it often consists of different types of components as shown in Figure 8.29.

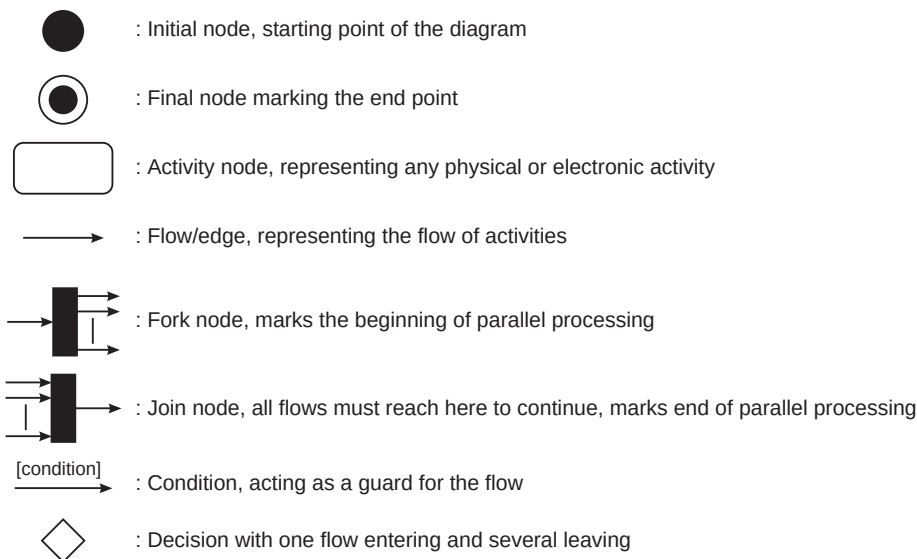


FIGURE 8.29 Notations in activity diagrams.

Figure 8.30 shows an example activity diagram for creating a user in a library system. The user fills up a form (hard copy) which is checked for correctness. If found correct, a screen is displayed to the user to enter details. A new user-name (id) and a password are also created.

8.5.2 Class Diagram

This is the main part of object-oriented analysis and design in UML. It can be used to show classes in the system, their interrelations, operations, and attributes. A *class* is shown as a rectangle with three compartments—*name* of the class, *attributes* and *operations*. For example, Figure 8.31 shows an example class *Door* with a single attribute *status* of type *String*, initialized to *closed*. It has two operations defined on it *Open()* and *Close()*.

An *object* is an instance of a class. A UML object is represented by a rectangle with one or more compartments like class. Figure 8.32 shows an object *D* of class *Door*.

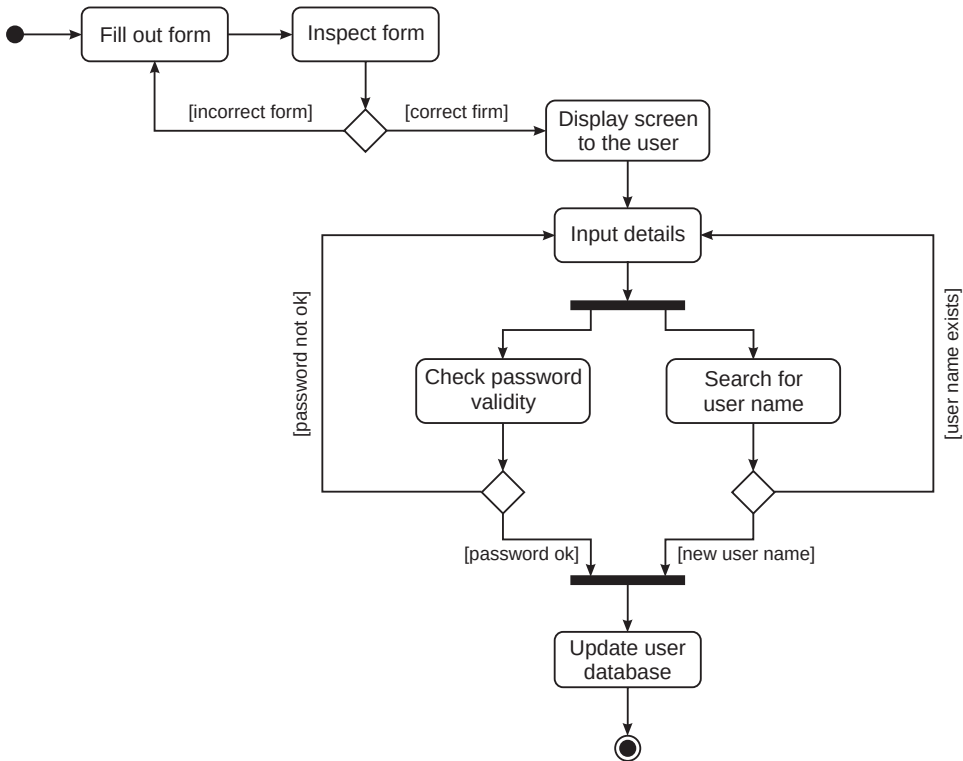


FIGURE 8.30 An activity diagram showing user creation in a library system.

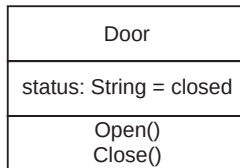


FIGURE 8.31 An example class.

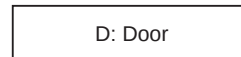


FIGURE 8.32 An example object.

Many kinds of relationships can be modelled in UML. These are as shown in Figure 8.33.

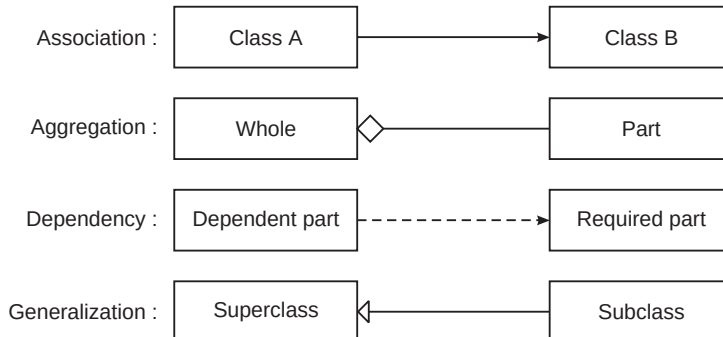


FIGURE 8.33 Relationships among UML classes.

An *association* defines a relationship between two or more classes resulting in *binary*- or *n*-ary relations respectively. *Aggregation* models the whole/part relationship. A *dependency* is a relationship that indicates that a model element is in some way dependent on another model element. *Generalization* is a relationship between a more general element (superclass) and a more specific element (subclass). It can be used to define *inheritance/hierarchy* as shown in Figure 8.34.

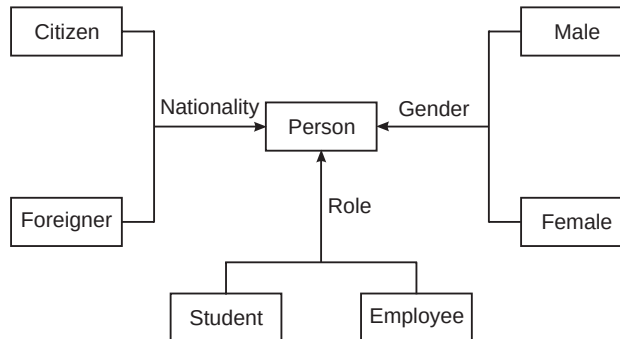


FIGURE 8.34 Inheritance among UML classes.

8.5.3 Communication/Collaboration Diagram

UML 2 *communication* diagrams (known as *collaboration* diagrams in UML 1.x) are used to show objects and messages involved in accomplishing a purpose or a set of purposes. The message sequence is identified by the sequence numbers. Format of a message is as follows: *precondition/sequence-number [expression] : return-value := message-name(parameter-list)*

Figure 8.35 shows a collaboration diagram showing the messages passed between a library user and the various components of the library system.

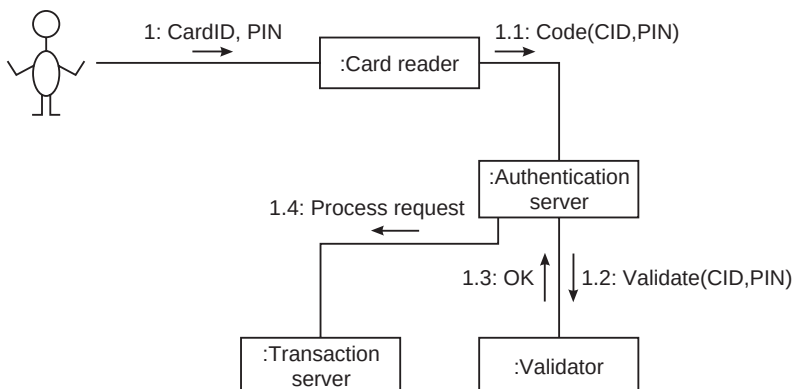


FIGURE 8.35 An example communication/collaboration diagram.

8.5.4 Component Diagram

Component diagrams depict how a system is split up into components. They also show the

dependency between the components. Figure 8.36 shows a component diagram with ports and interactions.

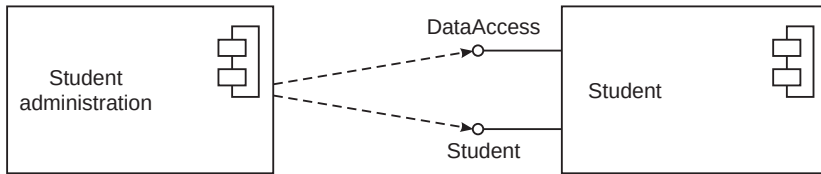


FIGURE 8.36 An example component diagram.

8.5.5 Use Case Diagram

These diagrams give an overview of the usage requirements for a system. They show relationships among *actors* and *use cases*. An *actor* is a role of an object or objects outside the system that interacts directly with it in a use case. It has class-like properties. A *use case* is a unit of functionality of the system or a class. There can be three different types of associations between actors and use cases as follows:

1. *Communicates*: Shows participation of an actor in a use case. It is shown as a solid line, and is the only possible relationship between actors and use cases.
2. *Extends*: It is a relationship from use case *A* to use case *B*, indicating that an instance of use case *B* may include the behaviour of use case *A*. It is shown as a generalization arrow labeled with `<< extends >>`.
3. *Uses*: It is also a relationship from use case *A* to use case *B* indicating that an instance of use case *A* will also include the behaviour specified by use case *B*. This is also shown as a generalization arrow labeled with `<< uses >>`.

Figure 8.37 shows an example of use case diagram.

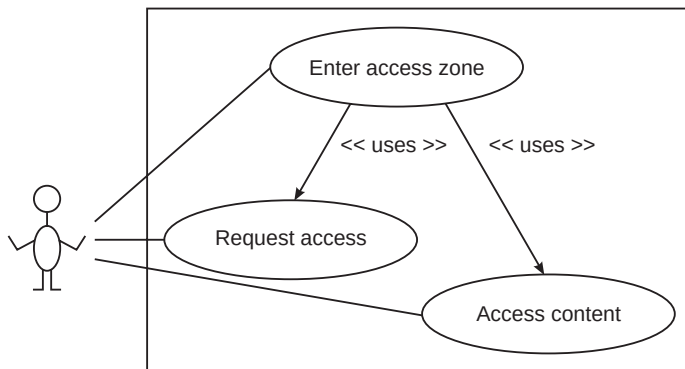


FIGURE 8.37 An example use case diagram.

8.5.6 Sequence Diagram

Sequence diagram of a system shows some simple interactions between objects arranged in a time-sequence. For each object, a life-line is shown. Exchange of messages between the objects

are also represented. When an object is not active, its lifeline is shown as a dashed line. During active period, the life-line is represented as a thin rectangular bar. An object may become active and inactive several times in its life-span. The life-span of an object may be terminated by putting a $\times$ at the end of a life-line. Messages are shown as arrows labeled with name of the message and parameters, if any. Figure 8.38 shows an example sequence diagram.

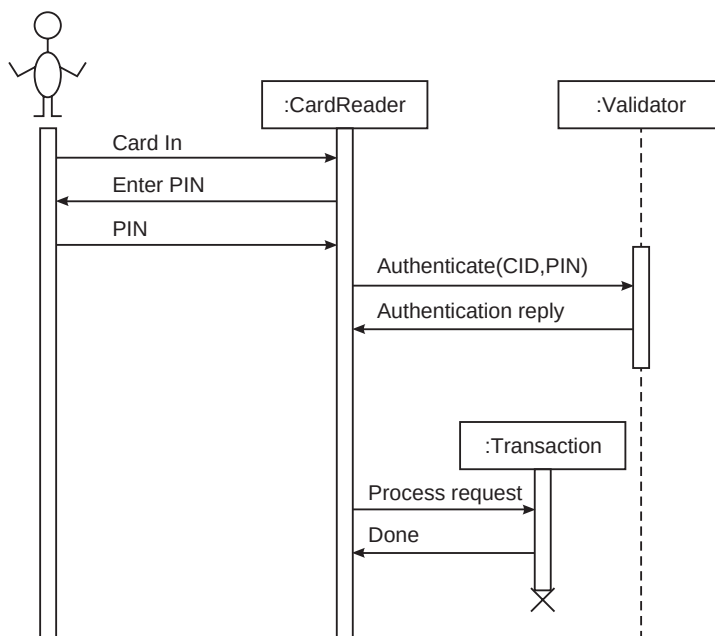


FIGURE 8.38 An example sequence diagram.

8.5.7 Other Diagrams

UML contains some more diagrams, which are not very much used in specifications, however, may be utilized to bring clarity to the specification. These are briefly described next:

1. *Composite structure diagram*: This is used to describe the internal structure of a class. It can include internal parts, ports, etc.
2. *Deployment diagram*: The *deployment diagram* of a system models the hardware used in system implementations, components deployed on the hardware and the association between these components.
3. *Interaction overview diagram*: These are variants of UML activity diagrams showing control flow. Nodes within this diagram are *frames*, rather than normal activities. Frames can depict any type of UML interaction diagrams (like sequence diagrams, communication diagrams, timing diagrams, etc.).
4. *Object diagram*: An object is an instance of a class. Object diagrams are similar to the class diagrams excepting the fact that these can be used to explain complex relationships between classes. Objects belonging to the same class may now be associated with objects of other classes in different ways.

5. *Package diagram*: Packages enable us to organize elements into groups. These can be used in conjunction with any of the UML diagrams just like file folders.
6. *State machine diagram*: This is similar to *StateCharts* discussed earlier. This can be used to capture state-based behaviour of a system.
7. *Timing diagram*: These can be used to explore the behaviour of one or more objects throughout a given period of time. The time may be represented as various phases of system operation in embedded system design.

8.5.8 UML Specification of an Elevator Control System

Let us consider a system to control elevators in a building with m floors. We need to move elevators between floors according to a set of constraints, as follows:

1. Each elevator has a set of m buttons, one for each floor. These can be pressed to cause the elevator to stop at that floor. The buttons also illuminate when pressed, and are turned off when the floor is reached.
2. Each floor excepting the first and top, has two buttons, one to request up, the other to request down. These buttons also illuminate when pressed and are turned off when an elevator visits that floor and moves in the desired direction.
3. In case there are no requests, an elevator remains at its current floor with door closed.

The detailed class diagram from the system has been shown in Figure 8.39. The collaboration diagrams have been shown in Figure 8.40(a) and (b) respectively. The sequence diagram has been shown in Figure 8.41.

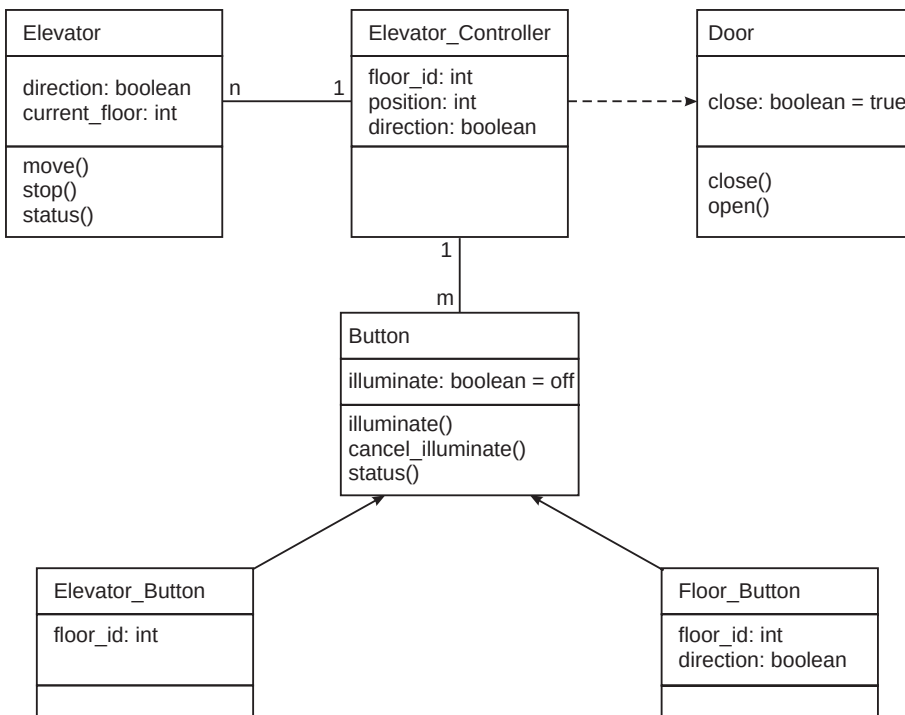


FIGURE 8.39

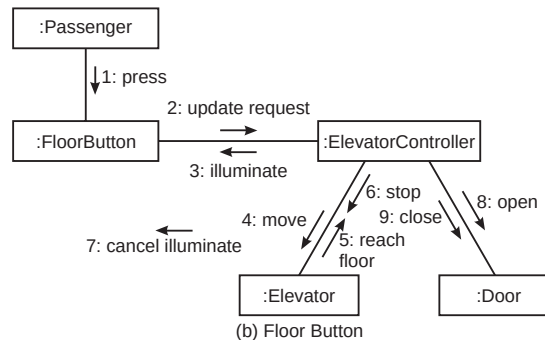
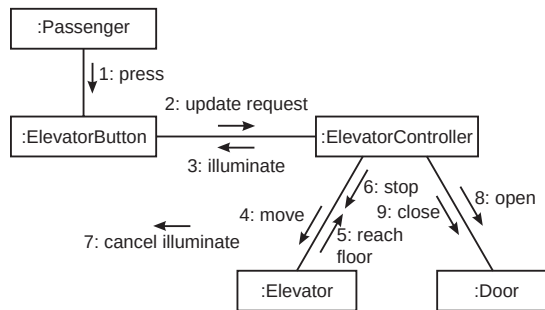


FIGURE 8.40 Collaboration diagrams for serving.

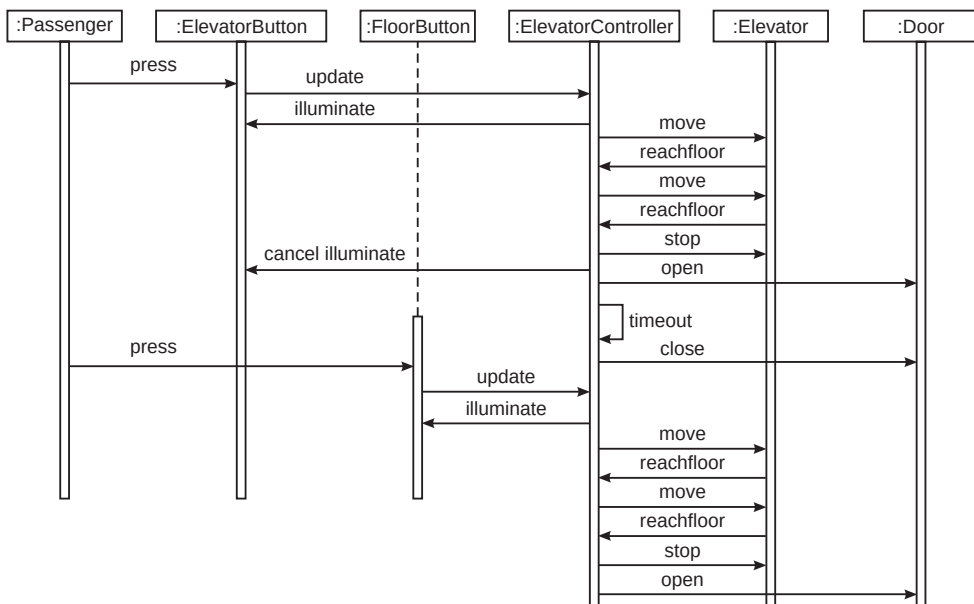


FIGURE 8.41 Sequence diagram for elevator operations.

8.6 CONCLUSION

Specification techniques help in expressing the desired system behaviour. In this chapter, we have seen many such techniques—some are very formal, while others are more pictorial. It may be noted that there exists many other specification techniques such as hardware description languages like Hardware C, VHDL, Verilog, etc. There also exist some system-level description languages, like, SystemC and System Verilog. From the specification of a system, the refinement (automated or manual) process identifies the modules to be realized in hardware or in software. In the next two chapters, we will see how to decide upon this partitioning and verify the partitioned design using hardware–software cosimulation.

EXERCISES

- 8.1 What is meant by a system specification? What are the desirable features of a specification technique?
- 8.2 What is a model of computation? Mention the drawbacks of Von Neumann model of computation in modelling real-time embedded systems.
- 8.3 State the differences between a finite state machine and a StateChart. Suppose, we have a sequence detector to detect the sequence “100011” in the input bit stream. The system outputs ‘1’ whenever the sequence is detected. However, if another input *a* is equal to ‘1’, it toggles output continually. Draw (i) the FSM for this system and (ii) the StateChart. From this explain how StateChart is superior to FSM.
- 8.4 Mention the features of a StateChart. In this light, describe which are advantageous for it and which are not.
- 8.5 What is meant by a superstate? What are its various types? In an ordinary FSM, what type of superstate is present?
- 8.6 What is meant by history mechanism in StateChart? How do you think can it be implemented in software?
- 8.7 Draw the StateCharts for sender and receiver processes of example Fig. 8.10.
- 8.8 Draw StateChart for the vending machine example of Fig. 8.23.
- 8.9 Draw StateChart for the traffic light controller of Fig. 8.20. Incorporate a *manual* button that if pressed will cause all the yellow lights to blink, indicating a manual mode of traffic control.
- 8.10 A system consists of two masters and four slave devices, connected over shared bus. A master normally executes the program by fetching it from memory. Whenever a slave device is ready to do a data transfer, it requests the appropriate master. The master in turn grants the request and the data transfer takes place till the slave raises the *over* signal. Upon getting a request from a device, a master first sends a request to the bus arbiter which arbitrates between the requests and grants the master to use the bus. Draw a StateChart based specification for the overall system.
- 8.11 Augment the StateChart of Fig. 8.8 for entering hour and minute also.
- 8.12 Draw the SDL specification for Exercise 8.8.
- 8.13 Draw the SDL specification for Exercise 8.9.